



CSCI 112

Programming with C

LABORATORY: 1

J. L. Pach

OUTLINE:

- ⌘ CLI
- ⌘ Toolchains etc.
- ⌘ Building a project
- ⌘ IDE
- ⌘ Hello World
- ⌘ Compiler vs Interpreter

SYLLABUS

SOME BASIC FACTS ABOUT THE COURSE

Course Name:

✂ Programming with C (CSCI 112)

Credit Hours:

✂ 3 credits

✂ 1 hour lecture twice a week

✂ 3 hours lab per week

Lecture (Mondays and Fridays)

✂ 12:00 – 12:50 PM.

✂ Science & Engineering Building
(S&E) 308

Lab (Mondays)

✂ 2:00 – 4:50 PM

✂ Engineering Lab/Classroom Building
(ELC) 315

SYLLABUS

- ✂ Course Description
- ✂ Textbooks
- ✂ Class Rules
- ✂ Grading
- ✂ Accommodations & Academic Dishonesty
- ✂ Declaration of authorship

COURSE DESCRIPTION

This course provides a comprehensive introduction to structured programming using the C language. Student will gain a deep understanding of memory management techniques such as pointers and dynamic allocation. The skills acquired in this course will be essential for those who wish to pursue further studies in languages like C++, C#, and Java, as well as microcontroller programming. Additionally, this course will lay a strong foundation for understanding computer architecture.

...A FEW WORDS

This course is designed for students in:

- ✂ Computer Science (CS),
- ✂ Software Engineering (SE),
- ✂ Electronics programs.

No prior programming experience is required — only basic computer literacy, such as using a web browser, email, and navigating the MS Windows environment.

TEXTBOOKS

Brian W. Kernighan, Dennis M. Ritchie. C Programming Language, 2nd Edition. Prentice Hall, 1988

 Dennis M. Ritchie

Dennis M. Ritchie

 ANSI

ANSI(C89) VS C99

- ⚡ Last year, the course was taught using the oldest and most widely adopted ANSI standard — C89. This choice was made because C89 does not include many of the conveniences introduced in later versions. While these conveniences make life easier for professional programmers, they can make it harder for beginners to fully grasp the fundamentals of programming in C.
- ⚡ The standard now used in our classes is **C99**, which is widely applied in embedded systems, especially in the field of electronics. This standard significantly lowers the entry barrier for beginners (for example, allowing variable declarations inside a for loop and supporting single-line comments `//`), while avoiding the complex and sometimes confusing features introduced in newer standards.

COURSE RULES

1. This course is an in-person course.
2. Attendance at every class is required.
3. Excused Absences. If there is any medical or any other kind of emergency, please let the instructor know immediately.
4. Makeup exams will only be given if you bring a valid medical documentation.
5. Each task without a sample required comment (below) is worth zero points.

GRADING

The course grade will be determined by two equally weighted components:

- ✂ the lecture component (40%)
- ✂ the laboratory component (60%).

The laboratory component will be evaluated based on three criteria:

- ✂ entrance quiz (30%),
- ✂ assignment (40%),
- ✂ brief concluding quiz (30%).

LABORATORY RULES

Entrance Quiz (optional):

- ✂ If included, the entrance quiz will consist of 3 questions. To pass, a student must score at least 2 points. Failed entrance quizzes may be retaken up to two times during the semester. Only failed quizzes can be retaken.

Assignment:

- ✂ Students will have 6 days to complete and submit their assignment.

Brief Concluding Quiz (optional):

- ✂ If included, this quiz serves as a summative assessment component, reinforcing the material covered during the lab session.

DETAILS

Accommodations:

- ✂ Students who need any type of accommodation should work with Montana Tech Disability Services and provide appropriate documentation as soon as possible.

Academic Dishonesty:

- ✂ You are encouraged to work in teams and use many resources including books and the Internet. However, each student must turn in his/her own work, and each student is responsible for understanding anything that is turned in. Refer to the Student Conduct Code for more information regarding plagiarism and cheating.

DECLARATION OF AUTHORSHIP

I acknowledge that I have worked on this assignment independently, except where explicitly noted and referenced. Any collaboration or use of external resources has been properly cited. I am fully aware of the consequences of academic dishonesty and agree to abide by the university's academic integrity policy. I understand the importance the consequences of plagiarism.

COMMENTS

There must be 4 lines of comments at the top of each source file. This heading should include your name, the class and semester, and the assignment number, and required statement. Source file without this comment gets **zero** points.

Template

```
//Jakub Pach  
//CSCI 112 Fall 2026  
//Programming Assignment #1  
//I acknowledge that I have worked on this assignment independently, except where explicitly noted and referenced...
```



Lecture & Laboratory:

(76078) CSCI 112 SEC 01 :PROGRAMMING WITH C

Check if you are registered for the course and have access to it!

INTRODUCTION

MEANING OF WORDS

Sometimes, when we look at the same thing, we see entirely different things. Therefore, to avoid misunderstandings, I will spend a considerable amount of time clarifying the nuances associated with translating the meanings of the concepts we will be using in our classes. All the material presented in this course will be of practical use.



6

COMPILER

1. A compiler takes the entire source code and translates it into a machine code file, often called **an executable**.
2. This executable file contains instructions that the computer's processor can directly execute.
3. Once compiled, the program can run independently without the need for the original source code or a compiler.

INTERPRETER

1. An interpreter translates the source code **line by line** as the program is running.
2. It doesn't create a separate executable file. Instead, it uses a virtual machine to execute the translated code.
3. The virtual machine provides an environment that mimics a real computer, allowing the program to run even if the underlying hardware architecture is different.

COMPILER VS INTERPRETER

Think of a compiler as a translator who translates an entire book from one language to another before you start reading it. An interpreter, on the other hand, is a translator who translates each sentence as you read it. A compiler translates the entire program at once, while an interpreter translates it line by line.

BIT & BYTE

BIT & BYTE

A **bit** is the smallest unit of data in a computer, representing a single binary value: either a **0** or a **1**.

A **byte** is a group of eight bits. A single byte can represent a wide range of values, such as a single character (like the letter '**A**' or the symbol '@') or an integer from **0** to **255**.

POSITIONAL NOTATION

POSITIONAL NOTATION

- ✂ Usually means the extension to any base (2, ..., 8,10,16 etc.) of the Hindu–Arabic numeral system (or decimal system).
- ✂ A positional system is a numeral system in which the contribution of a digit to the value of a number is the value of the digit multiplied by a factor determined by the position of the digit.
- ✂ In modern positional systems, such as the decimal system, the position of the digit means that its value must be multiplied by some value: in 444, the three identical symbols represent four hundreds, four tens, and four units, respectively, due to their different positions in the digit string.

BASE OF THE NUMERAL SYSTEM BASE

- ✂ In mathematical numeral systems the radix r is usually the number of unique digits, including zero, that a positional numeral system uses to represent numbers.
- ✂ The highest symbol of a positional numeral system usually has the value one less than the value of the radix of that numeral system. The standard positional numeral systems differ from one another only in the base they use.
- ✂ The radix is an integer that is greater than 1, since a radix of zero would not have any digits, and a radix of 1 would only have the zero digit.

BINARY NUMERAL SYSTEM BASE

$$A = \sum_{i=0}^{n-1} R^i \cdot d_i$$

$$\overset{2}{1} \overset{1}{2} \overset{0}{3}_{(10)} = (10^2 \cdot 1) + (10^1 \cdot 2) + (10^0 \cdot 3)$$

$$\overset{7}{0} \overset{6}{1} \overset{5}{1} \overset{4}{1} \overset{3}{1} \overset{2}{0} \overset{1}{1} \overset{0}{1}_{(2)} = (2^7 \cdot 0) + (2^6 \cdot 1) + (2^5 \cdot 1) + (2^4 \cdot 1) + (2^3 \cdot 1) + (2^2 \cdot 0) + (2^1 \cdot 1) + (2^0 \cdot 1) = 123_{(10)}$$

CONVERTING DECIMAL NUMBERS TO BINARY

Steps:

1. Divide the decimal number by 2. The remainder of the division will be either 0 or 1.
2. If the remainder is 0, write down a 0.
3. If the remainder is 1, write down a 1.
4. Repeat steps 2 and 3 until the decimal number is 0.
5. Read the numbers **from bottom to top** to get the binary number.

EXAMPLE

123_{10}

✂ $123 / 2 = 61$ (remainder 1)

✂ Write down a 1.

✂ $61 / 2 = 30$ (remainder 1)

✂ Write down a 1.

✂ $30 / 2 = 15$ (remainder 0)

✂ Write down a 0.

✂ $15 / 2 = 7$ (remainder 1)

✂ Write down a 1.

✂ $7 / 2 = 3$ (remainder 1)

✂ Write down a 1.

✂ $3 / 2 = 1$ (remainder 1)

✂ Write down a 1.

✂ $1 / 2 = 0$ (remainder 1)

✂ Write down a 1.

CONVERTING BINARY NUMBERS TO OCT

1111011₂

001.111.011 = 1.7.3

0111.1011 = 7.B

Radix	Value	Binary Value	Symbol
Octal	0	0000	0
Octal	1	0001	1
Octal	2	0010	2
Octal	3	0011	3
Octal	4	0100	4
Octal	5	0101	5
Octal	6	0110	6
Octal	7	0111	7
Hexadecimal	8	1000	8
Hexadecimal	9	1001	9
Hexadecimal	10	1010	A
Hexadecimal	11	1011	B
Hexadecimal	12	1100	C
Hexadecimal	13	1101	D
Hexadecimal	14	1110	E
Hexadecimal	15	1111	F

UNSIGNED REPRESENTATION

 USR

SIGN-MAGNITUDE REPRESENTATION



DATA IN A COMPUTER

can essentially be stored using two standards:

1. integers represented in binary,
2. real (floating-point) numbers stored according to the IEEE 754 standard.

Everything else is a combination or interpretation based on these two fundamental forms of representation.

IDE

Integrated development environment

INTEGRATED DEVELOPMENT ENVIRONMENT

An Integrated Development Environment (IDE) is a comprehensive software application that consolidates the essential tools needed for software development into a single graphical user interface. It typically combines:

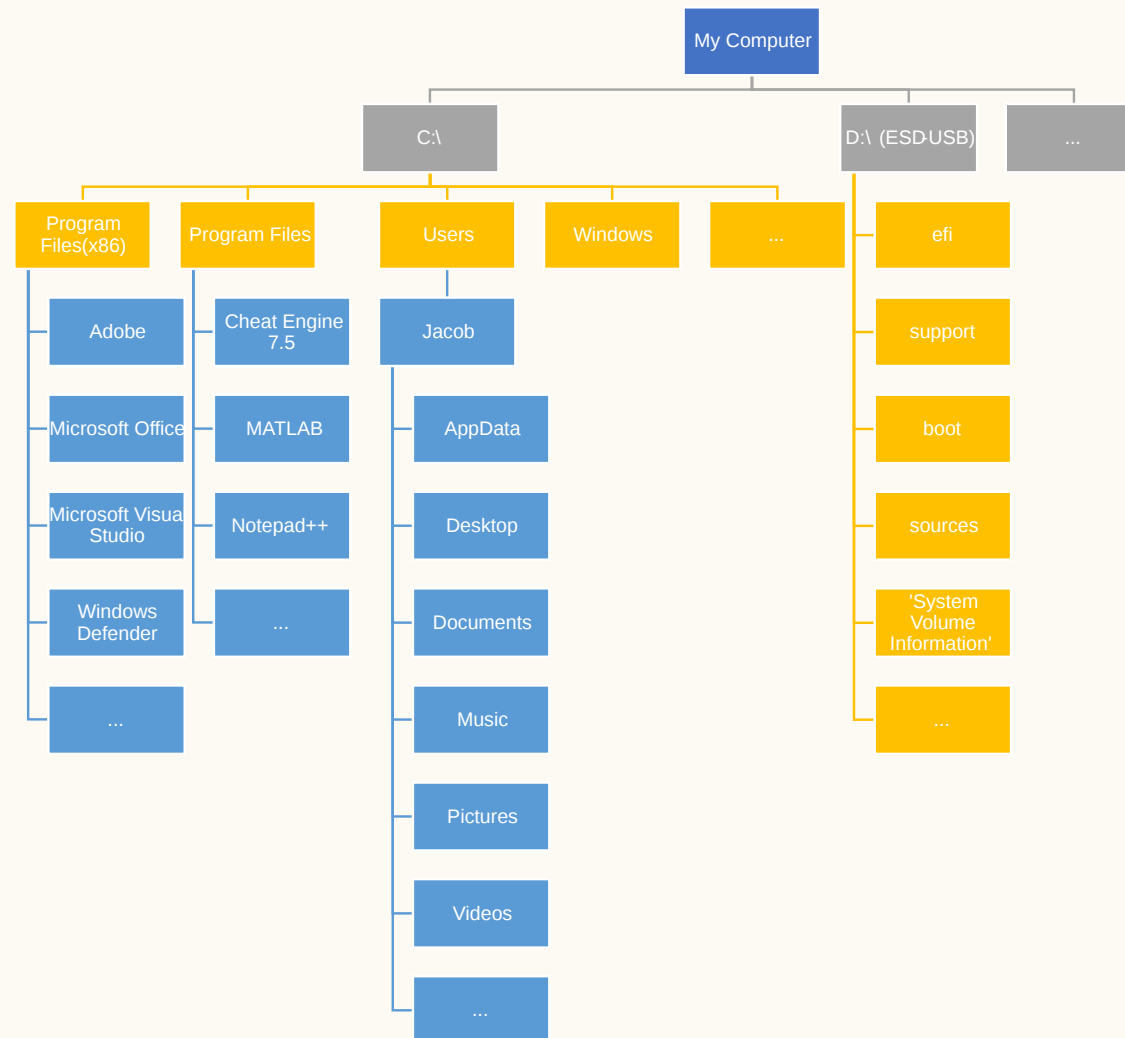
1. a source **code editor** for writing code
2. a **compiler** and **linker** to transform that code into an executable program.
3. a built-in **debugger** to help developers identify and resolve logical errors

efficiently during the programming process.

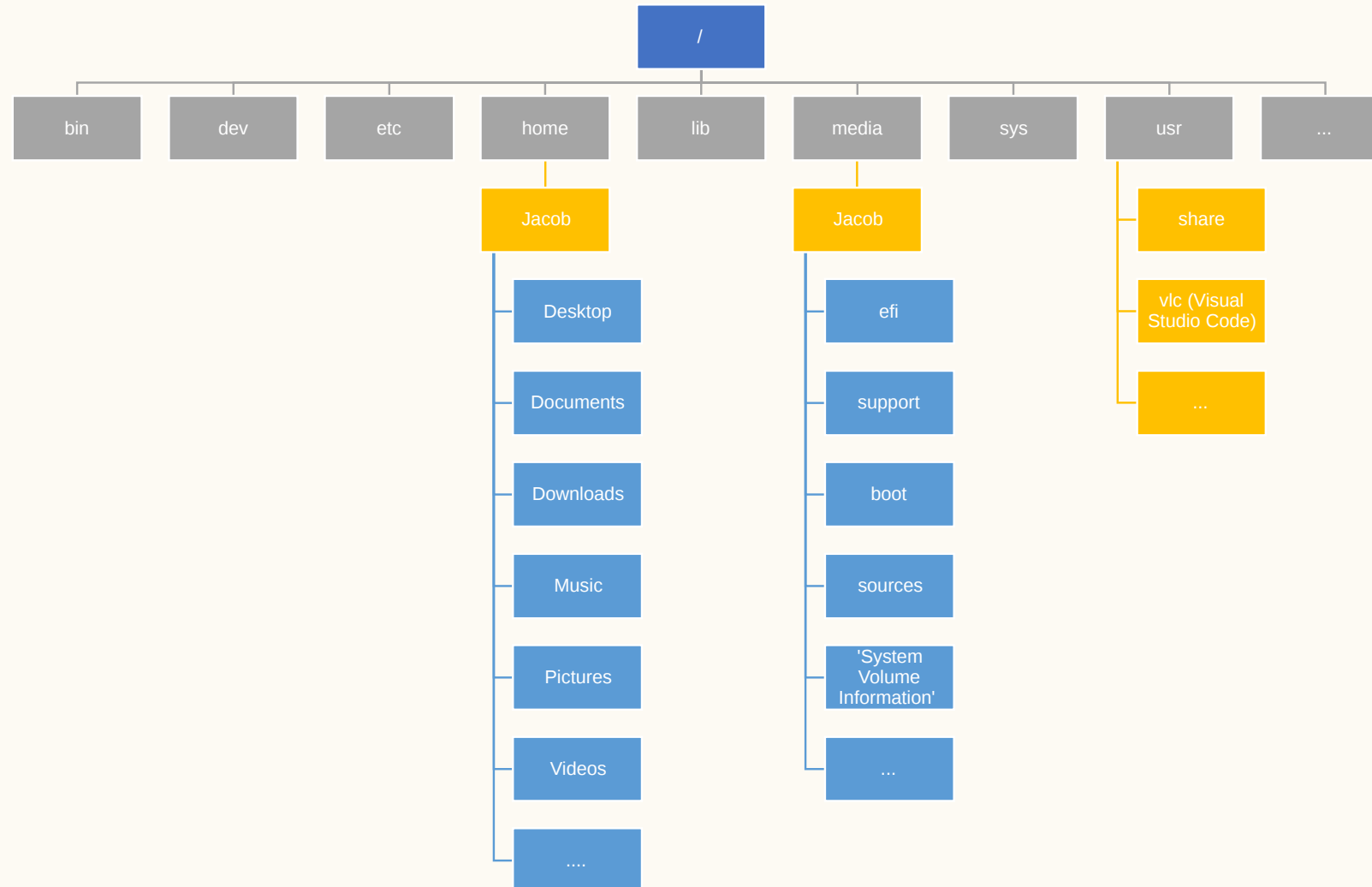
INTEGRATED DEVELOPMENT ENVIRONMENT



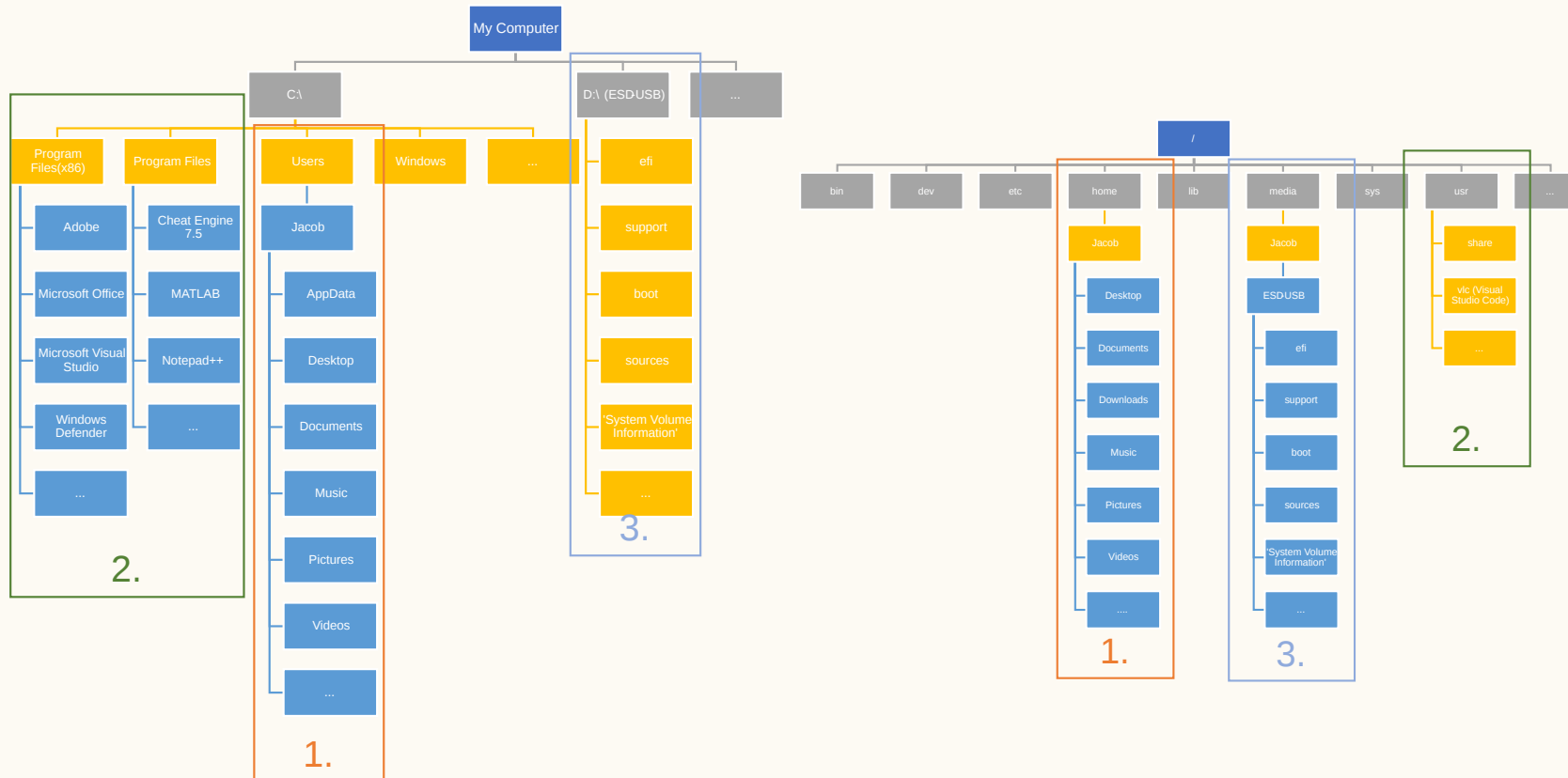
FILE SYSTEM HIERARCHY - WINDOWS



FILE SYSTEM HIERARCHY - LINUX



FILE SYSTEM HIERARCHY - COMPARISON



1. Files of user , 2. Apps of user, 3. External devices e.g. pendrive

DIFFERENCES IN FILE HIERARCHY

Windows - My Computer concept:

- ✕ Provides a graphical view of all available drives and partitions, including the system drive (usually C:), external drives, network drives, etc.
- ✕ System drive (C:):
 - ✕ Typically, the main drive where the Windows operating system and most programs are installed.
- ✕ Peripherals:
 - ✕ Each peripheral device (e.g., USB drive, external hard drive) is usually visible as a separate entity in "My Computer".

DIFFERENCES IN FILE HIERARCHY

Linux

Root directory (/):

- ✂ This is the starting point for the entire file hierarchy. All files and directories are located directly or indirectly within the root directory.

Directory hierarchy:

- ✂ The directory structure is more consistent and logical. Each directory has a specific purpose (e.g., /etc for configuration files, /home for user home directories).

Peripheral devices in /media or /mnt:

- ✂ When a peripheral device is connected, the system automatically creates a mount point for it in the /media or /mnt directory.

SUMMARY OF DIFFERENCES

Feature	Windows	Linux
Main concept	"My Computer"	Root directory (/)
System drive	C:	Usually no direct equivalent, system files are scattered across various directories
Peripherals	Visible as separate entities	Mounted in /media or /mnt
Structure	Less formal, more visual	Formal, hierarchical, based on conventions

SUMMARY OF DIFFERENCES

Why are there such differences?

- ✖ **History:** Windows was created as an operating system for home computers, with a more user-friendly interface. Linux, on the other hand, has its roots in the server environment, where consistency and efficiency are more important.
- ✖ **Philosophy:** Linux places a greater emphasis on standards and consistency, as reflected in its file hierarchy. Windows, on the other hand, offers a more intuitive interface that may be easier for novice users.

CLI

(Command-Line Interface)

CLI

A command-line interface (CLI), also known as a command-line shell, is a means of interacting with software via commands – each formatted as a line of text. The concept of interacting with a computer via text evolved over two decades, transitioning from physical hardware to sophisticated software.

```
Windows PowerShell
```

```
Copyright (C) Microsoft Corporation. All rights reserved.
```

```
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows
```

```
PS C:\Users\Jacob> cd .\Desktop\
```

```
PS C:\Users\Jacob\Desktop> ls
```

```
PS C:\Users\Jacob\Desktop>
```

NAVIGATING THE COMMAND LINE

Windows vs. Linux

WINDOWS:

- ✂ **Cmd:** The most commonly used command to open the command prompt.
To obtain the username and device name, we use the whoami command.

LINUX:

- ✂ **Terminal:** A generic term for the window where commands are typed.
To obtain the current directory path, we use the pwd command.

NOTES:

- ✂ **Up/down arrows:** Browse command history.
- ✂ **Tab:** Autocompletes filenames, directories, and commands.

NAVIGATING THE COMMAND LINE: WINDOWS

From the Start menu:

1. Click the Start button in the lower left corner of the screen.
2. Type `cmd` and press Enter.

From the Run dialog box

1. Simultaneously press the Windows key and R.
2. In the opened window, type `cmd` press Enter.

```
Microsoft Windows [Version 10.0.22631.4602]
(c) Microsoft Corporation. All rights reserved.
C:\Users\Jacob> whoami
notebooki7\Jacob
C:\Users\Jacob>
```

NAVIGATING THE COMMAND LINE: LINUX

From the Start menu:

1. Click the Start button in the lower left corner of the screen.
2. Click the Accessories.
3. Click the Terminal.

Keyboard shortcut:

1. The most commonly used shortcut is Ctrl+Alt+T.

```
jacob@raspberrypi:~ $ pwd
/home/jacob
jacob@raspberrypi:~ $
```


DIFFERENCES IN COMMAND PROMPT MESSAGES

Between Windows and Linux

WINDOWS

- ✂ **Default message:** Typically displays the full path to the currently open directory.
- ✂ **Example:** `C:\Users\User\Documents>`
- ✂ **Purpose:** Provides the exact location of the user within the file system hierarchy.

LINUX

- ✂ **Default message:** Most often presents the username and hostname (device name).
- ✂ **Example:** `user@computer:~$`
- ✂ **Purpose:** Informs about the currently logged-in user and the machine they are working on.

BASIC NAVIGATION AND DIRECTORY MANAGEMENT

Windows:

- ✂ `dir`: Lists files and directories in the current directory.
- ✂ `cd`: Changes the current directory.
- ✂ `cd..`: Moves up one directory level.

Linux:

- ✂ `ls`: Lists files and directories in the current directory.
- ✂ `cd`: Changes the current directory.
- ✂ `cd ..`: Moves up one directory level.

Notes:

- ✂ Both `dir` and `ls` are used to list directory contents in both different systems.
- ✂ In Linux, `cd ..` **requires a space between the dots, Windows does not.**

EXAMPLE 1 - DIR, LS

`dir`: Lists files and directories in the current directory.

```
C:\Users\Jacob>dir
12/19/2024 12:14 PM <DIR>      .
09/05/2023 09:06 AM <DIR>      ..
10/05/2022 11:55 PM <DIR>      Contacts
01/04/2025 08:29 AM <DIR>      Desktop
09/07/2024 08:51 AM <DIR>      Documents
01/03/2025 05:47 PM <DIR>      Downloads
                4 File(s)      818 bytes
                42 Dir(s) 62,825,164,800 bytes free

C:\Users\Jacob>ls
ls
'ls' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\Jacob>
```

`ls`: Lists files and directories in the current directory.

```
jacob@raspberrypi:~ $ ls
Bookshelf Desktop Documents Downloads Music
Pictures Public Templates Videos
jacob@raspberrypi:~ $ dir
Bookshelf Desktop Documents Downloads Music
Pictures Public Templates Videos
jacob@raspberrypi:~ $
```

EXAMPLE 2 - `CD`, `CD..`

Windows

✂ `cd` : Changes the current directory.

✂ `cd..` : Moves up one directory level.

```
C:\Users\Jacob>cd Desktop
C:\Users\Jacob\Desktop>cd..
C:\Users\Jacob>cd Desktop
C:\Users\Jacob\Desktop>cd ..
C:\Users\Jacob>cd /
C:\>
```

Linux

✂ `cd` : Changes the current directory.

✂ `cd ..` : Moves up one directory level.

```
jacob@raspberrypi:~ $ cd Desktop
jacob@raspberrypi:~/Desktop $ cd ..
jacob@raspberrypi:~ $ cd Desktop
jacob@raspberrypi:~/Desktop $ cd..
-bash: cd..: command not found
jacob@raspberrypi:~/Desktop $ cd /
jacob@raspberrypi:/ $ pwd
/
jacob@raspberrypi:/ $
```

TEXT MANIPULATION AND OUTPUT

Windows:

`echo`: Displays a message on the screen.

Linux:

`echo`: Displays a message on the screen.

Note: Both `echo` commands have similar functionality, but there might be slight variations in options and behavior.

EXAMPLE 3 - ECHO

`echo` : Displays a message on the screen.

Windows

```
C:\Users\Jacob>echo Hello World!
Hello World!
C:\Users\Jacob> echo "Hello World!"
"Hello World!"
C:\Users\Jacob>echo.

C:\Users\Jacob>echo
ECHO is on.
C:\Users\Jacob>
```

Linux

```
jacob@raspberrypi:~ $ echo Hello World!
Hello World!
jacob@raspberrypi:~ $ echo "Hello World!"
Hello World!
jacob@raspberrypi:~ $echo

jacob@raspberrypi:~ $echo.
-bash: echo.: command not found
jacob@raspberrypi:~ $
```

REDIRECTION

```
echo Hello, world!
```

To redirect output to a file: `echo Hello, world! > myfile.txt`

Note: Redirection works similarly in both systems, using the `>` symbol to overwrite a file and `>>` to append to a file.

```
C:\Users\Jacob>echo Hello world! > myfile.txt
```

```
C:\Users\Jacob>
```

EXAMPLE 4 - ECHO

- ✂ The simplest way to create a new file.
- ✂ Redirection works similarly in both systems, using the `>` symbol to overwrite a file and `>>` to append to a file.

myfile.txt

Hello world

CREATING AND DELETING FILES AND DIRECTORIES

Windows:

`copy` : Copies files.

`del` : Deletes files.

`md` : Makes a new directory.

Linux:

`cp` : Copies files.

`rm` : Removes files or directories.

`mkdir` : Makes a new directory.

EXAMPLE 5 - COPY, DEL, MD --- CP, RM, MKDIR

Windows

```
C:\Users\Jacob>md Folder
C:\Users\Jacob>dir
01/04/2025 10:06 AM <DIR>      Folder
07/16/2023 07:09 AM <DIR>      Music
...
C:\Users\Jacob> cd Folder
C:\Users\Jacob\Folder> echo. > empty.txt
C:\Users\Jacob\Folder> copy empty.txt copyEmpty.txt
...
01/04/2025 12:22 PM <DIR>      .
01/04/2025 10:08 AM <DIR>      ..
01/04/2025 12:19 PM          3 copyEmpty.txt
01/04/2025 12:19 PM          3 empty.txt
...
C:\Users\Jacob\Folder> del copyEmpty.txt
C:\Users\Jacob\Folder>
```

Linux

```
jacob@raspberrypi:~ $ mkdir Folder
jacob@raspberrypi:~ $ ls
Bookshelf Desktop Documents Downloads Folder
...
jacob@raspberrypi:~ $ cd Folder
jacob@raspberrypi:~/Folder $ echo > empty.txt
jacob@raspberrypi:~/Folder $ cp empty.txt copyEmpty.txt
...
jacob@raspberrypi:~/Folder $ ls
copyEmpty.txt empty.txt
...
jacob@raspberrypi:~/Folder $ rm copyEmpty.txt
jacob@raspberrypi:~/Folder $
```

EXECUTING COMPILED PROGRAMS

Windows:

- ✂ To run a compiled program, you usually just type the program name, followed by the `.exe` extension: `program_name.exe`.
- ✂ Windows will automatically add the `.exe` extension if it is omitted.

Linux:

- ✂ To run a compiled program, you typically use the following syntax: `./program_name`.
- ✂ The `./` part specifies that the program should be executed in the current directory.

EXAMPLE 6 - PING

Windows

```
C:\Users\Jacob>cd C:\Windows\System32
C:\Windows\System32>ping.exe www.google.com

Pinging www.google.com [172.217.14.228] with 32 bytes of data:
Reply from 172.217.14.228: bytes=32 time=16ms TTL=113
Reply from 172.217.14.228: bytes=32 time=16ms TTL=113
Reply from 172.217.14.228: bytes=32 time=16ms TTL=113
Reply from 172.217.14.228: bytes=32 time=16ms TTL=113

Ping statistics for 172.217.14.228:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 16ms, Maximum = 16ms, Average = 16ms

C:\Windows\System32>
```

Linux

```
jacob@raspberrypi:/usr/bin $ ./ping www.google.com

PING www.google.com (172.217.14.228) 56(84) bytes of data.
64 bytes from sea30s02-in-f4.1e100.net (172.217.14.228): icmp_seq=1 ttl=115 time=16.2 ms
64 bytes from sea30s02-in-f4.1e100.net (172.217.14.228): icmp_seq=2 ttl=115 time=16.0 ms
64 bytes from sea30s02-in-f4.1e100.net (172.217.14.228): icmp_seq=3 ttl=115 time=16.1 ms
64 bytes from sea30s02-in-f4.1e100.net (172.217.14.228): icmp_seq=4 ttl=115 time=16.0 ms
64 bytes from sea30s02-in-f4.1e100.net (172.217.14.228): icmp_seq=5 ttl=115 time=16.0 ms
^C
--- www.google.com ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4006ms
rtt min/avg/max/mdev = 16.013/16.068/16.192/0.069 ms

jacob@raspberrypi:/usr/bin $
```

ADDITIONAL TOOLS

Clear the console

✂ To clear the console/terminal window, use the appropriate command, such as

Windows: `cls` **Linux:** `clear`

Rename file

✂ Instead of copying a file to a new name, you can use a command to rename the file.

The syntax is identical to that of copying. **Windows** `rename` **Linux** `mv`

Help

✂ To obtain additional assistance on using the application, please type the program

name followed by a space and **Windows:** `/h`, `-h` **Linux:** `--h`

IP ADDRESS

An IP address is like a unique mailing address for every device connected to the internet. It allows computers to locate and communicate with each other. Think of it as a numerical label assigned to each device, making it possible for data to be sent to the correct destination.

Windows: `ipconfig`

Linux: `ip addr`

EXAMPLE 7 - IP

Windows

```
C:\Users\Jacob>ipconfig

...
Ethernet adapter Ethernet 4:

    Connection-specific DNS Suffix  . : butte.campus
    Link-local IPv6 Address . . . . . : fe80::813a:b45d:4cd4:63fb%26
    IPv4 Address. . . . . : 10.38.32.232
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 10.38.32.254

...
C:\Users\Jacob>
```

Linux

```
jacob@raspberrypi:~ $ ip addr
...
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
qdisc pfifo_fast state UP group default qlen 1000
    link/ether b8:27:eb:dc:64:a3 brd ff:ff:ff:ff:ff:ff
    inet 10.38.32.256/24 brd 10.38.32.255 scope global dynamic noprefixroute eth0
        valid_lft 525094sec preferred_lft 438694sec
    inet6 fe80::d8c5:56d0:e193:bd13/64 scope link
        valid_lft forever preferred_lft forever
...
jacob@raspberrypi:~
```

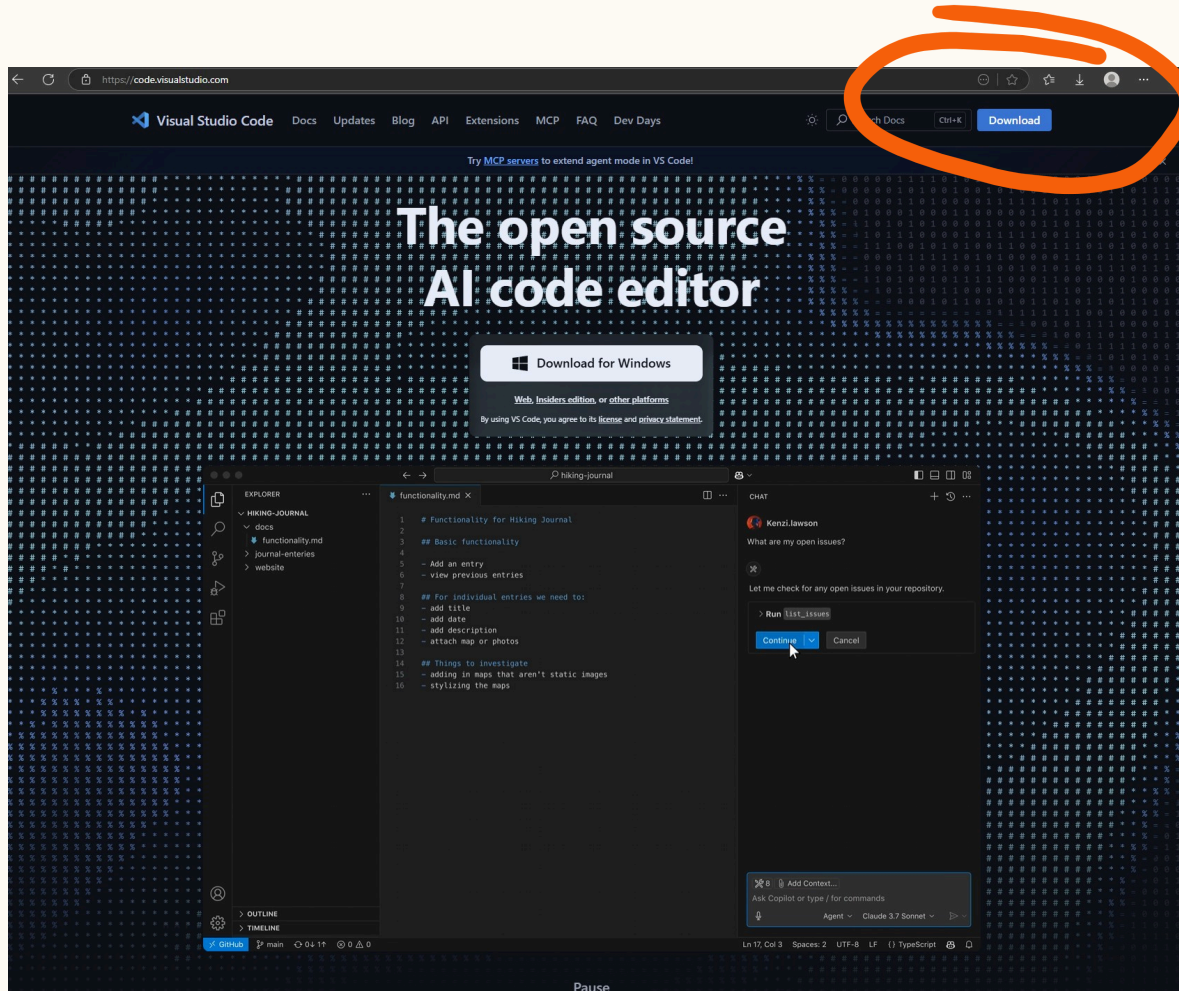

TEMPLATE CODE – DOWNLOAD & UNZIP

Not here! On your personal computer

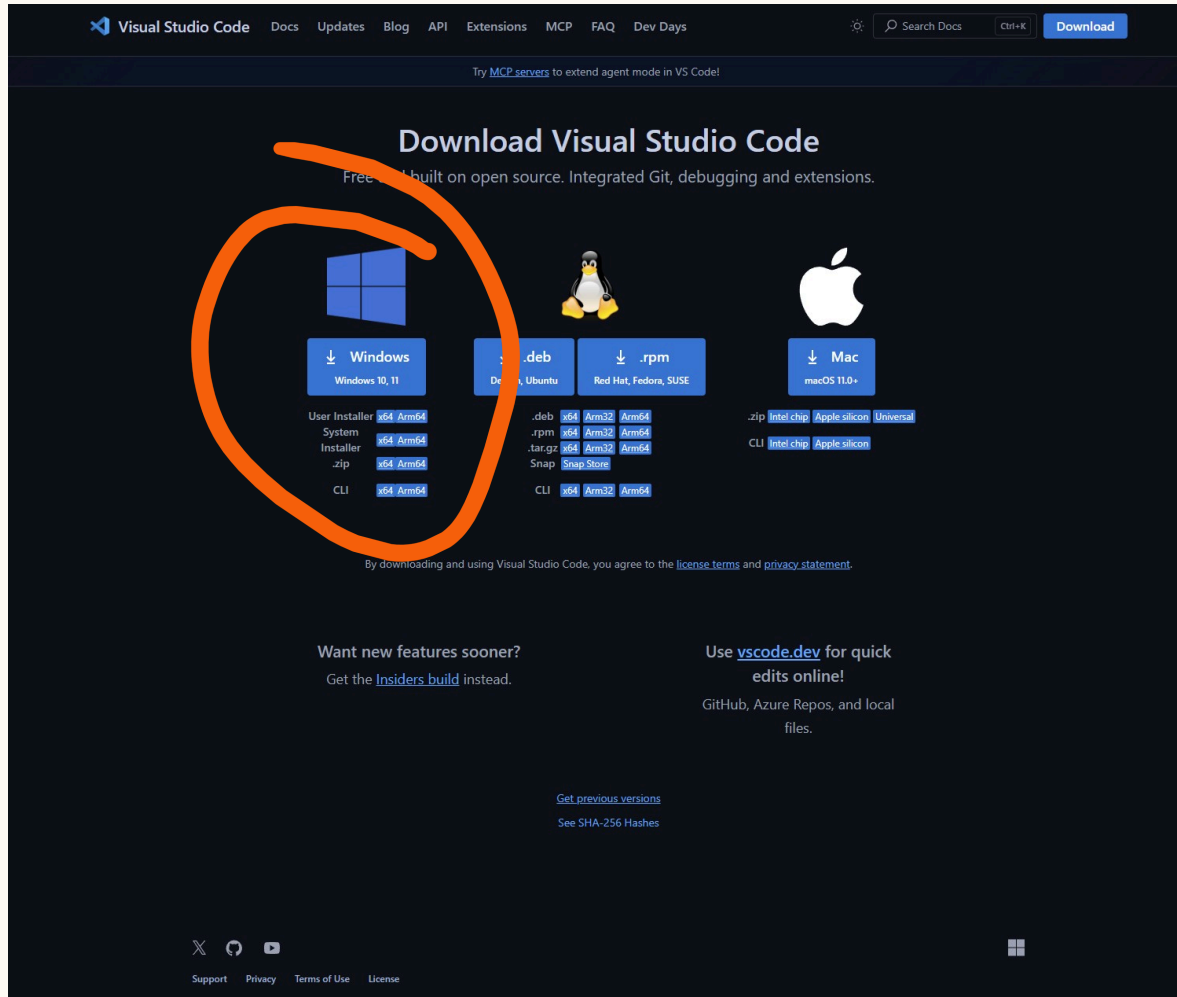
Please do not install VSC on university computers. Everything should already be installed. You only need to check the VSC extensions.

| HOW TO GET VC

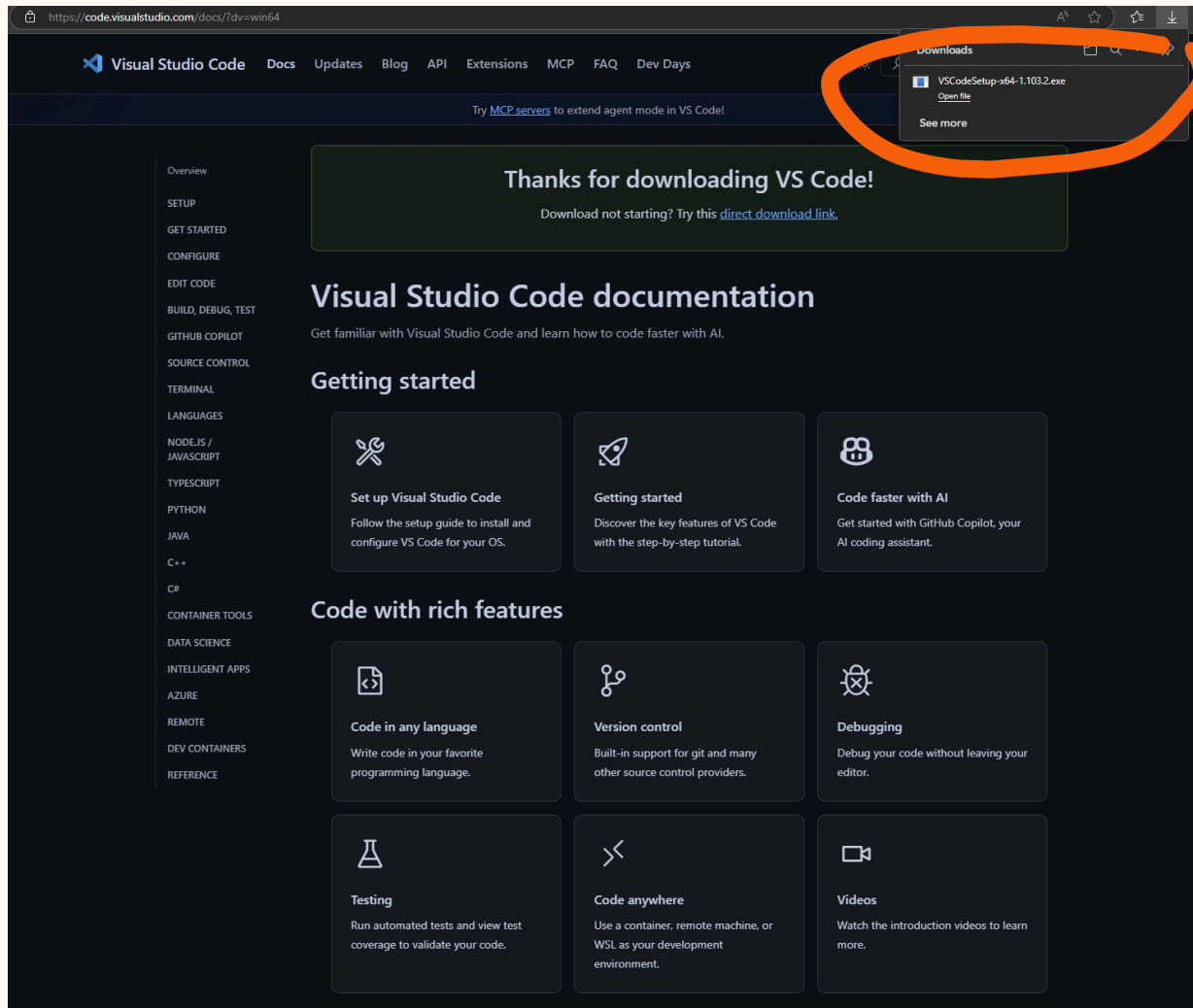
GO TO: [HTTPS://CODE.VISUALSTUDIO.COM/](https://code.visualstudio.com/)



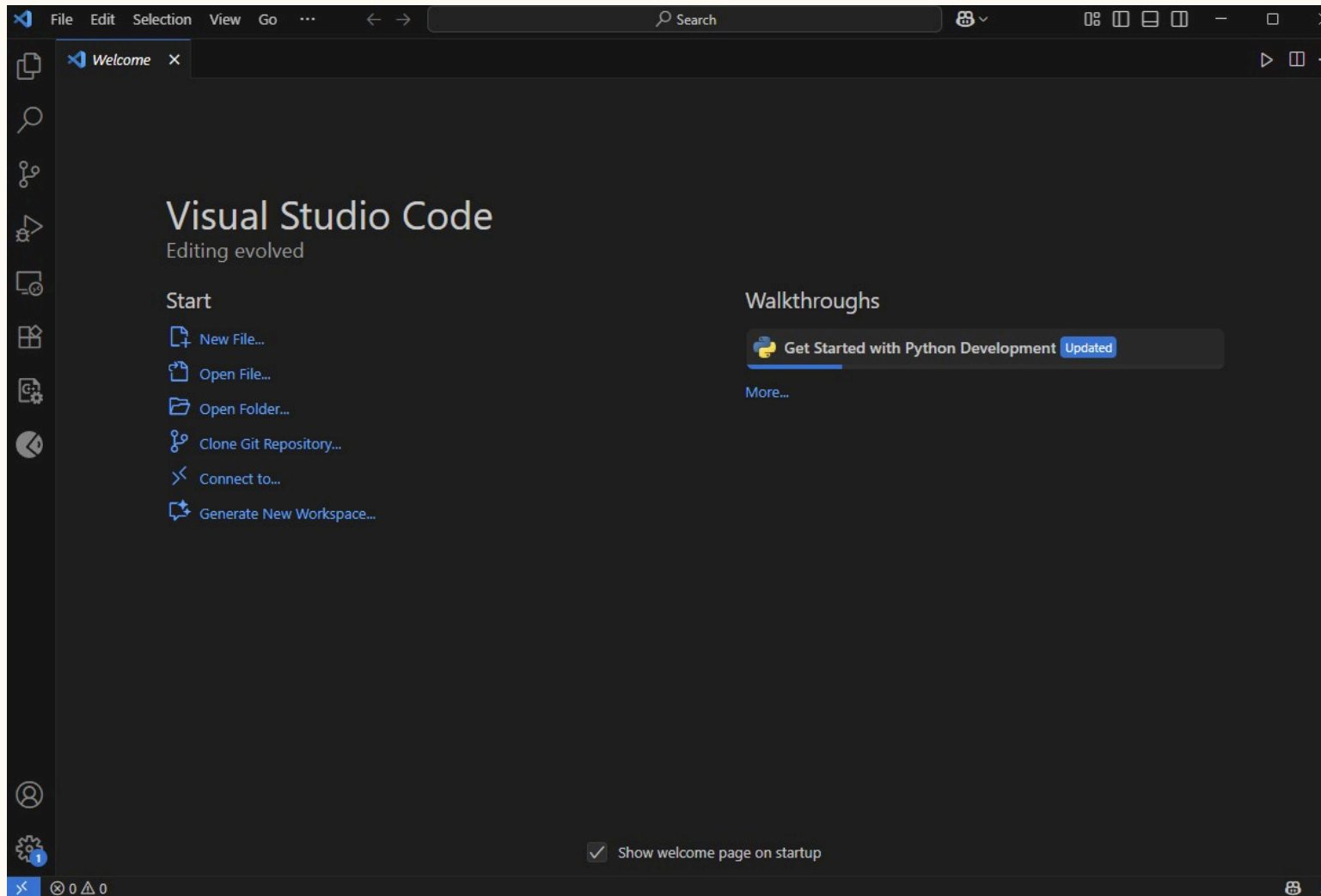
SELECT SYSTEM INSTALLER X64:



DOWNLOAD & INSTALL: VSCODESETUP-X64.EXE

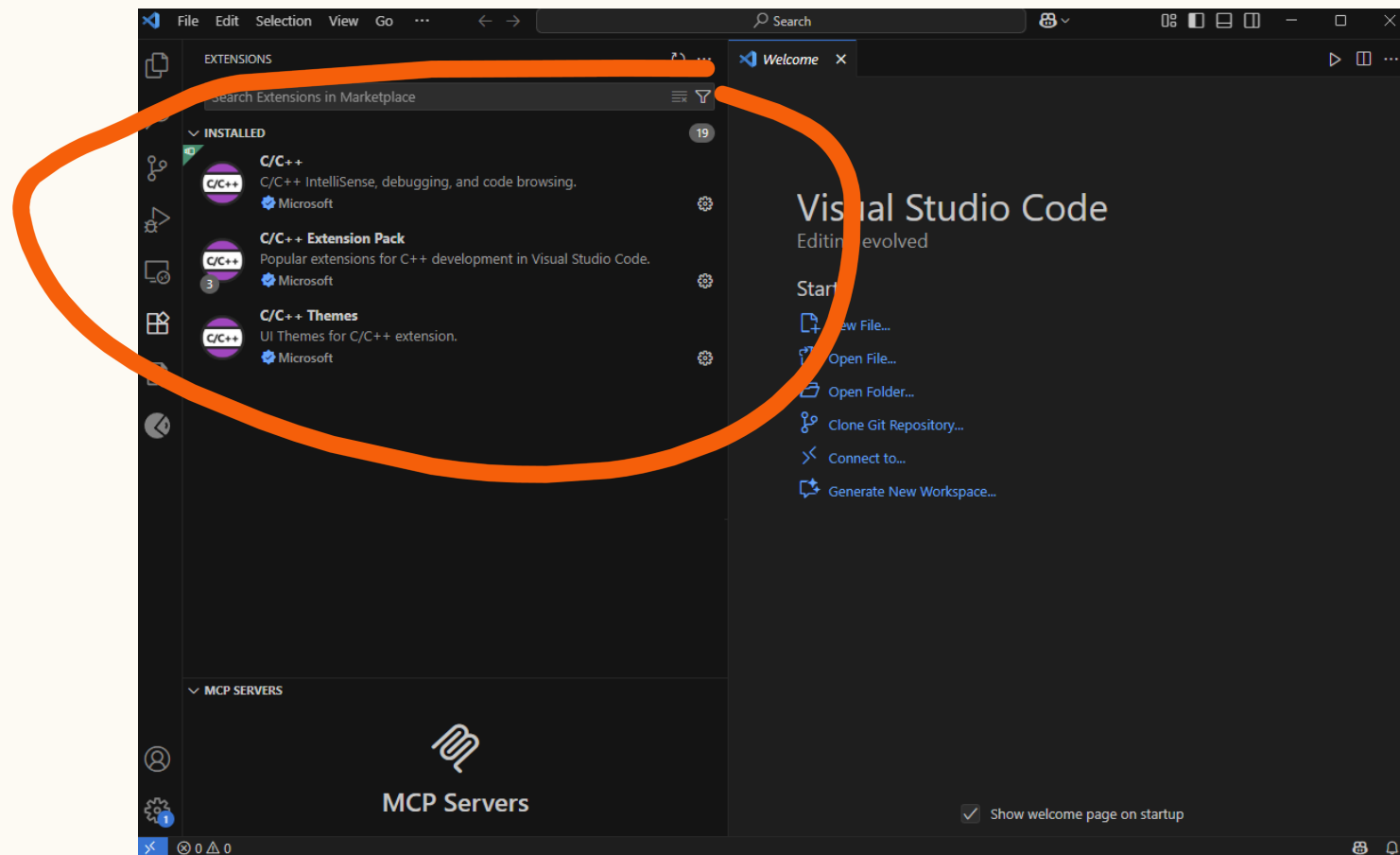


INSTALL EXTENSIONS

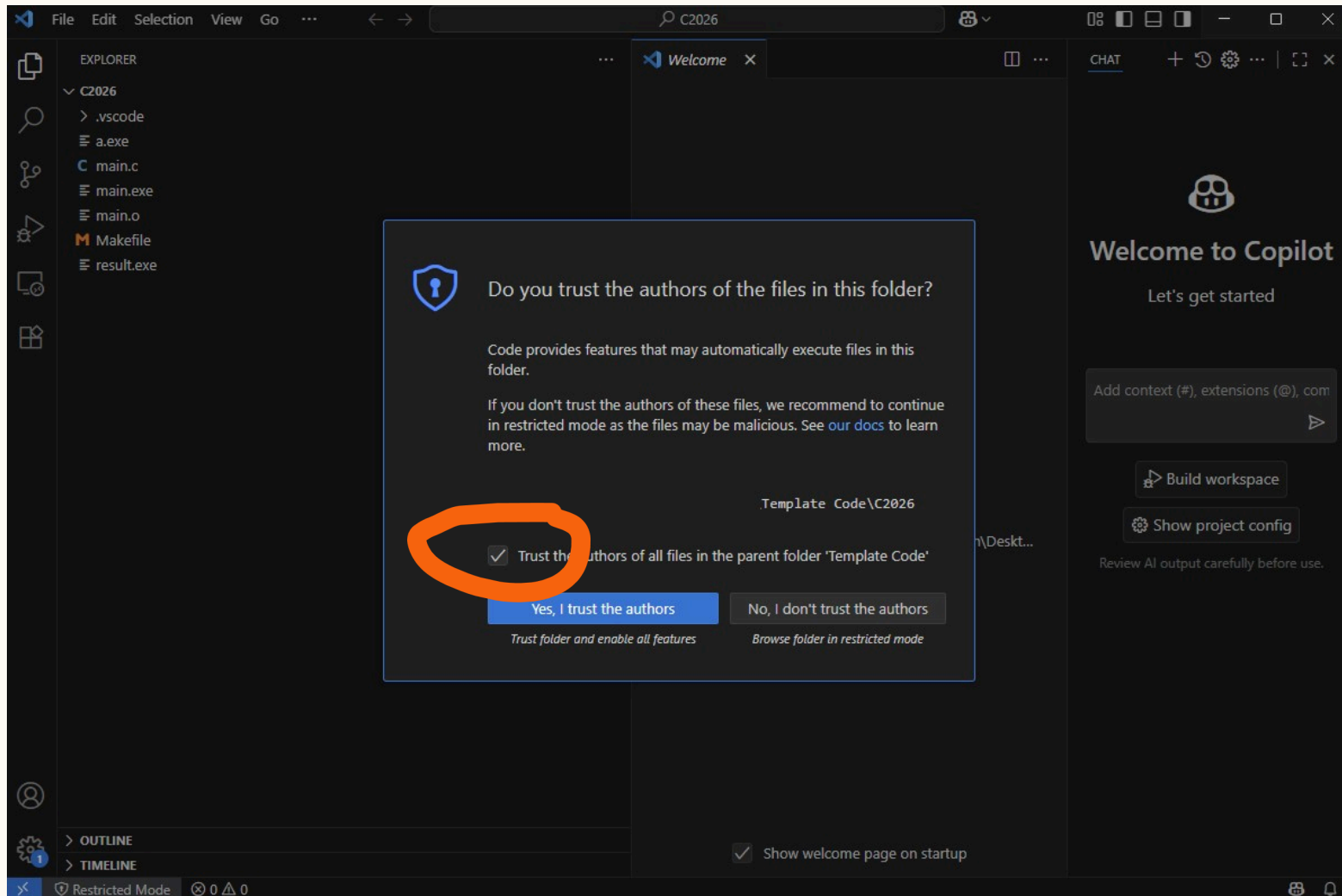


INSTALL EXTENSIONS

C/C++; C/C++ Extension Pack; C/C++ Themes



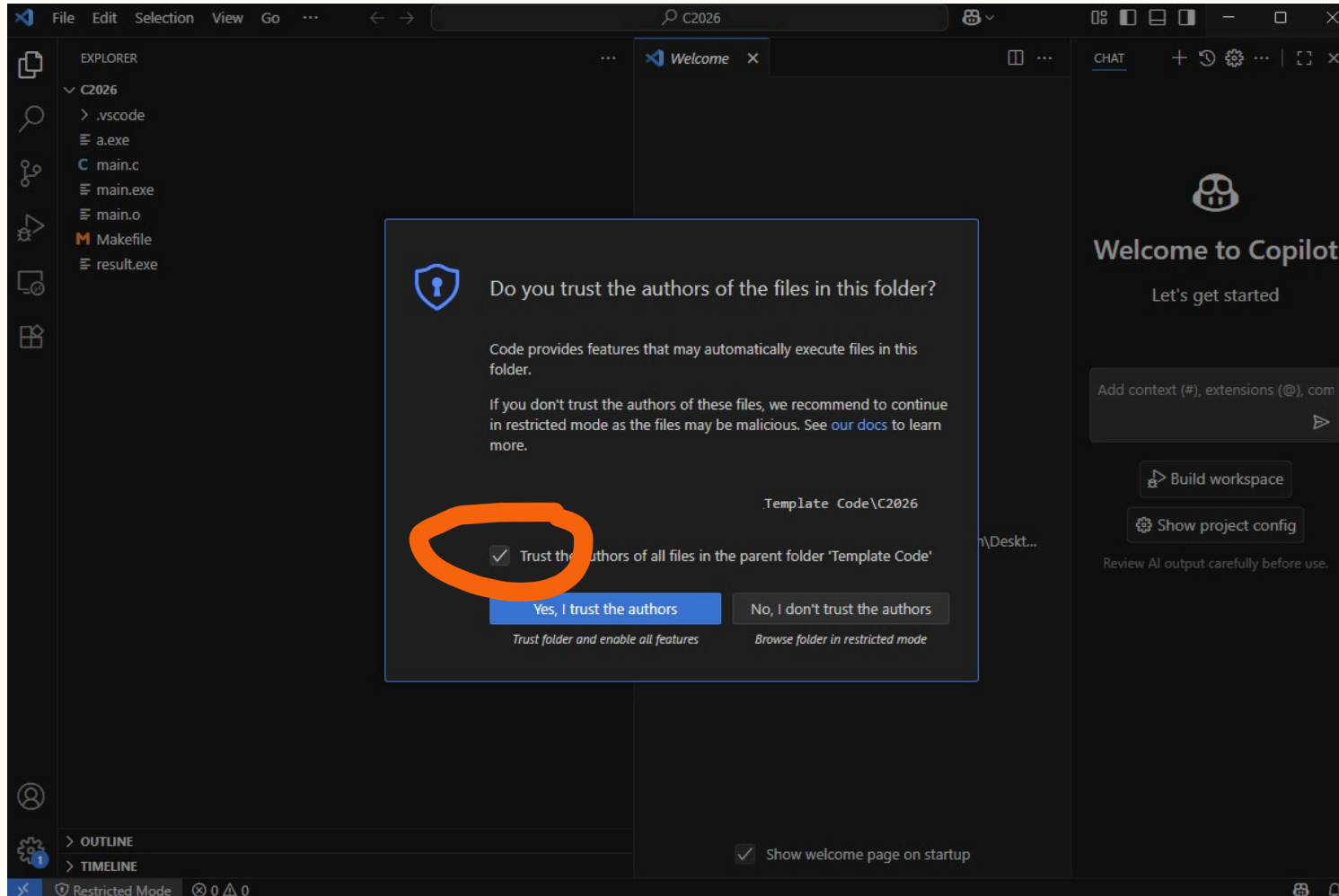
OPEN TEMPLATE CODE



READ YOUR FIRST C PROGRAM – MAIN.C

```
1 // #define clearBuffer() while (getchar() != '\n');
2 #include <stdio.h>
3 #include <stdbool.h>
4
5 int main()
6 {
7     char string[12] = "Hello world";
8     printf("%s", string);
9
10    getchar();
11    return 0;
12 }
13
```

RUN YOUR FIRST C PROGRAM



THANK

You